

Numerical Addendum: EEQ Calculator

Generated/Reviewed Documentation (Codex/Opus/Human) for v0.0.1

The EEQ calculator turns one conformation into two scalar vectors. The supporting work is the smooth coordination-number count and the construction of the dense Coulomb matrix. The numerical centre is the $N \times N$ Cholesky factorization of that matrix and the bordered-system reduction that enforces the total charge without factoring the indefinite augmented matrix. It is the only $O(N^3)$ dense step in this calculator; the coordination count and matrix fill are $O(N^2)$, and the two solves reuse the same triangular factor. The listings below are distilled from the source — loops and bookkeeping trimmed — but the constants, guards, inequalities, and array shapes are the implementation's.

1 Smooth coordination counts

The code first moves positions from Å into a_0 and looks up one D4 parameter row per atom. In the coordination-number pass, $cn(i)$ is the accumulated smooth neighbour count for atom i . Each unordered atom pair i, j is visited once. If its squared distance is greater than $cn_cutoff_bohr_sq$, the pair is absent from both counts. Otherwise the code evaluates one number,

$$c_{ij} = \frac{1}{2} \operatorname{erfc} \left[k_{CN} \left(\frac{R_{ij}}{r_{cov,i} + r_{cov,j}} - 1 \right) \right],$$

and adds that same c_{ij} to $cn(i)$ and $cn(j)$. R_{ij} is the pair distance in a_0 , r_{cov} is the covalent radius from params, and $k_{CN} = ee_cn_steepness = 7.5$. The comparison is strict, so a pair landing exactly at the cutoff is still counted.

The complementary error function replaces a discontinuous integer neighbour count by a smooth transition. At $R_{ij} = r_{cov,i} + r_{cov,j}$ the argument is zero, so the pair contributes $1/2$. Shorter pairs contribute more than $1/2$ and approach one as the argument becomes negative; longer pairs contribute less than $1/2$ and approach zero. The shape is differentiable in R_{ij} for counted pairs. The configured 25.0 Å cutoff remains a hard cutoff: a pair beyond it contributes exactly zero, with no reweighting of the remaining pairs.

```
double cn_cutoff_bohr = cn_cutoff * BOHR_PER_ANGSTROM;
double cn_cutoff_bohr_sq = cn_cutoff_bohr * cn_cutoff_bohr;

Eigen::VectorXd cn = Eigen::VectorXd::Zero(N);
for (size_t i = 0; i < N; ++i) {
    for (size_t j = i + 1; j < N; ++j) {
        double dx = pos(i,0) - pos(j,0);
        double dy = pos(i,1) - pos(j,1);
        double dz = pos(i,2) - pos(j,2);
        double dist_bohr_sq = dx*dx + dy*dy + dz*dz;
        if (dist_bohr_sq > cn_cutoff_bohr_sq) continue;

        double dist_bohr = std::sqrt(dist_bohr_sq);
        double rcov_sum = params[i].rcov + params[j].rcov;
        double arg = cn_steepness * (dist_bohr / rcov_sum - 1.0);
        double count = 0.5 * std::erfc(arg);

        cn(i) += count;
        cn(j) += count;
    }
}

for (size_t i = 0; i < N; ++i)
    chi_eff(i) = params[i].chi
        + params[i].kappa * std::sqrt(cn(i) + 1e-14);
```

The second loop turns `cn(i)` into the effective electronegativity χ_i^{eff} held in `chi_eff(i)`: a base electronegativity χ_i (`params[i].chi`) shifted by the coordination through κ_i (`params[i].kappa`), the per-element sensitivity of electronegativity to neighbour count. The 10^{-14} term is the source guard carried from `dftd4` arithmetic: an isolated atom still takes a square root of a positive number. The guard changes the zero-coordination value by $\kappa_i 10^{-7}$ in Ha.

2 Dense Coulomb matrix

A is a dense real $N \times N$ matrix. Filling it is an $O(N^2)$ all-pairs pass. Its diagonal stores a local quadratic charge cost,

$$A_{ii} = \eta_i + \frac{\sqrt{2/\pi}}{r_{g,i}},$$

where η_i is `params[i].gam` and $r_{g,i}$ is `params[i].rad`. The first term is the chemical hardness. The second term is the Gaussian self-Coulomb term. Both are positive for every project-local protein element row and for the fallback row.

For $i \neq j$, $A(i, j)$ is the Ohno–Klopman pair kernel

$$A_{ij} = \gamma_{ij}(R_{ij}) = \frac{1}{\sqrt{R_{ij}^2 + 1/(\eta_i \eta_j)}}.$$

The same value is written into $A(j, i)$, so the stored matrix is symmetric. At zero separation the off-diagonal value is finite, $\gamma_{ij}(0) = \sqrt{\eta_i \eta_j}$; at long range it approaches the atomic-unit Coulomb factor $1/R_{ij}$.

```
Eigen::MatrixXd A(N, N);
for (size_t i = 0; i < N; ++i) {
    A(i, i) = params[i].gam + SQRT_2_OVER_PI / params[i].rad;

    for (size_t j = i + 1; j < N; ++j) {
        double dx = pos(i,0) - pos(j,0);
        double dy = pos(i,1) - pos(j,1);
        double dz = pos(i,2) - pos(j,2);
        double dist_bohr_sq = dx*dx + dy*dy + dz*dz;

        double inv = 1.0 / (params[i].gam * params[j].gam);
        double gamma = 1.0 / std::sqrt(dist_bohr_sq + inv);
        A(i, j) = gamma;
        A(j, i) = gamma;
    }
}
```

The self term is the diagonal contribution from a charge’s finite-width Coulomb energy. Without it, identical atoms at zero separation would have $A_{ii} = \eta_i$ and $A_{ij} = \eta_i$, so the diagonal would only tie one contact off-diagonal. Adding $\sqrt{2/\pi}/r_{g,i}$ raises the Gershgorin row centre above that contact value. For the H, C, N, O, S, and fallback rows in `PhysicalConstants.h`, A_{ii} is larger than every zero-distance off-diagonal involving atom i . Gershgorin’s theorem locates every eigenvalue inside a disc — one per row, centred on that row’s diagonal entry A_{ii} and reaching out by the sum of the row’s off-diagonal magnitudes. When the full row also satisfies $A_{ii} > \sum_{j \neq i} |A_{ij}|$, Gershgorin discs lie on the positive real axis; because A is symmetric, its eigenvalues are then positive, and the matrix is symmetric positive-definite. The implementation does not separately test the row sums. It uses the factorization call below as the SPD gate.

3 Cholesky factorization

Eigen's LLT call factors A as $A = LL^T$, with L lower triangular. The factorization is the $O(N^3)$ step in this calculator. Once L exists, each solve with a right-hand side b is two triangular substitutions: first solve $Ly = b$ by forward substitution, then solve $L^T x = y$ by back-substitution. That returns $x = A^{-1}b$ to the accuracy of the floating-point factor and substitutions.

The code uses Cholesky rather than a general linear solve because this formulation is intended to supply a symmetric positive-definite block. In exact arithmetic, an SPD matrix gives positive pivots and no row exchanges are needed. In this implementation, `chol.info()` is the numerical gate: if Eigen cannot complete the positive-definite factorization on the supplied A , the calculator returns no result.

```
Eigen::LLT<Eigen::MatrixXd> chol(A);
if (chol.info() != Eigen::Success) {
    OperationLog::Error("EqResult",
        "Cholesky decomposition failed (N=" + std::to_string(N) +
        ") -- Coulomb matrix not positive definite");
    return nullptr;
}

Eigen::VectorXd A_inv_chi = chol.solve(chi_eff);
Eigen::VectorXd ones = Eigen::VectorXd::Ones(N);
Eigen::VectorXd A_inv_ones = chol.solve(ones);
```

The same factorization serves both right-hand sides. A_inv_chi is u , the solution of $Au = \chi^{\text{eff}}$. A_inv_ones is v , the solution of $Av = 1$. The cost of forming L is paid once; the second right-hand side reuses the same triangular factors.

4 Charge constraint

The constrained first-order system is bordered:

$$\begin{bmatrix} A & \mathbf{1} \\ \mathbf{1}^T & 0 \end{bmatrix} \begin{bmatrix} q \\ \lambda \end{bmatrix} = \begin{bmatrix} -\chi^{\text{eff}} \\ Q \end{bmatrix}.$$

Q is Q_{total} , defaulting to 0 e, and λ is the scalar multiplier for the charge sum. The augmented matrix is indefinite because its lower-right block is zero and the constraint row has no quadratic charge cost. Positive-definiteness is a bowl, curving upward in every direction; the zero lower-right block gives the multiplier λ a direction with no curvature of its own, tipping the bordered system into a saddle — up some ways, down others. That saddle is what *indefinite* names, and it is why Cholesky, which needs the bowl, cannot factor it. The code avoids factoring that matrix. It eliminates against the SPD block A .

From the first block row,

$$q = -A^{-1}\chi^{\text{eff}} - \lambda A^{-1}\mathbf{1} = -(u + \lambda v).$$

Applying the second block row $\mathbf{1}^T q = Q$ gives

$$\lambda = -\frac{Q + \mathbf{1}^T u}{\mathbf{1}^T v}.$$

Thus the two Cholesky solves build the two vectors used by the constraint: u is the unconstrained response to electronegativity, and v is the response to the all-ones vector in the first block row. The multiplier chooses how much of v must be mixed in so that the entries of q sum to Q .

```

double denom = ones.dot(A_inv_ones);          //  $1^T v$ 
if (std::abs(denom) < 1e-30) {
    OperationLog::Error("EeqResult",
        "charge_constraint_denominator_is_zero");
    return nullptr;
}

double lambda =
    -(Q_total + ones.dot(A_inv_chi)) / denom;

Eigen::VectorXd charges =
    -(A_inv_chi + lambda * A_inv_ones);

double charge_residual = charges.sum() - Q_total;
charges.array() -= charge_residual / static_cast<double>(N);

```

denom is $1^T v$. If it is smaller than 10^{-30} in magnitude, the scalar constraint solve is treated as unusable and the calculator returns no result; equality with 10^{-30} proceeds. Otherwise the algebraic reduction gives charges. In exact arithmetic, those charges satisfy both the stationarity equations and the charge constraint. In floating point, the two triangular solves can leave a nonzero scalar charge residual; the code does not estimate $\kappa(A)$. The last two lines measure that residual and subtract the same constant from every atom. That uniform shift makes the pre-clamp sum equal to Q , up to rounding, and preserves every charge difference $q_i - q_j$. It does not re-solve the stationarity equations.

Stored charges then pass through the output clamp. If $|q_i| > \text{eeq_charge_clamp}$, with the configured value $2.0e$, the stored atom value is set to $\pm 2.0e$ and a geometry-choice record is written. A charge with $|q_i| = \text{eeq_charge_clamp}$ is stored unchanged. The clamp fires after the uniform charge correction, so the stored sum can move away from Q . The reported charge statistics are computed from the post-clamp atom values.

```

for (size_t i = 0; i < N; ++i) {
    auto& atom = conf.MutableAtomAt(i);
    atom.eeq_cn = cn(i);

    double qi = charges(i);
    if (std::abs(qi) > charge_clamp) {
        qi = (qi > 0) ? charge_clamp : -charge_clamp;
        ++n_clamped;
    }
    atom.eeq_charge = qi;
}

```

5 Output shape

EEQ emits scalars, not tensors. It does not call `SphericalTensor::Decompose`, and it does not populate the shared $[T_0, T_1, T_2]$ tensor layout. A partial charge is a scalar attached to an atom, but it is not the T_0 part of a shielding tensor.

```

eeq_charges.npy // (N,) float64, atom.eeq_charge in elementary charges
eeq_cn.npy      // (N,) float64, atom.eeq_cn coordination number

```

For $N = 0$, Compute returns `nullptr` before any matrix is allocated. For $N > 0$, `WriteFeatures` on a successful result calls `NpyWriter::WriteFloat64` for both arrays with one entry per atom.

6 Glossary

Smooth coordination number. A tally with a soft rim — it fades a neighbour out across a band of distance instead of clicking from one to zero at a fixed radius (a neighbour count with a smooth pair weight before the cutoff: close atoms count nearly one, distant counted atoms count nearly zero, and the change between them is continuous). In this code, a pair at $R_{ij} = r_{\text{cov},i} + r_{\text{cov},j}$ contributes exactly $1/2$ to both atoms before the total is used in $\chi_i + \kappa_i \sqrt{\text{CN}_i + 10^{-14}}$. Formally, it is the pair sum $\text{CN}_i = \sum_{j \neq i} c_{ij}$ with $c_{ij} = \frac{1}{2} \text{erfc}(k_{\text{CN}}(R_{ij}/(r_{\text{cov},i} + r_{\text{cov},j}) - 1))$ for counted pairs.

Ohno–Klopman kernel. The $1/R$ Coulomb curve with its spike at contact filed down to a finite plateau (regularized to a finite zero-distance value, the plateau height set by the two chemical hardness parameters). At zero separation it is $\sqrt{\eta_i \eta_j}$, while at large separation it tends to $1/R_{ij}$; the EEQ matrix writes this same value into both symmetric off-diagonal slots. Formally, $\gamma_{ij}(R) = (R^2 + 1/(\eta_i \eta_j))^{-1/2}$.

Cholesky factorization. A symmetric positive-definite matrix split into a triangle times its own transpose (a lower triangular L with $A = LL^T$), so one dense factorization leaves a pair of triangular solves for each right-hand side. Here LLT forms $A = LL^T$, then the two right-hand sides χ^{eff} and $\mathbf{1}$ reuse that same L . Formally, Cholesky applies to symmetric positive-definite A and solves $Ax = b$ by $Ly = b$ followed by $L^T x = y$.

Bordered-system reduction. A square block with one extra row and column framing it as a border (an augmented linear system whose extra row and column enforce a scalar constraint). Rather than factor the whole bordered matrix, the code solves the inner block twice and recovers the multiplier with one scalar division. The EEQ code never factors the $(N + 1) \times (N + 1)$ KKT matrix; it solves for $u = A^{-1} \chi^{\text{eff}}$ and $v = A^{-1} \mathbf{1}$, then combines them with one scalar λ . Formally, the reduction is $\lambda = -(Q + \mathbf{1}^T u)/(\mathbf{1}^T v)$ and $q = -(u + \lambda v)$.

Conditioning. The amount a solve magnifies input and rounding errors on the way from b to x . In this calculator an ill-conditioned A makes u , v , and the scalar sum of q more sensitive to roundoff; the code measures only the final charge-sum residual and removes it by a uniform shift. Formally, the relevant scale is the condition number of A , which bounds the relative sensitivity of $Ax = b$ to perturbations in A and b .